Intl. Trans. in Op. Res. xx (2020) 1–24

A Mixed Integer Linear Programming Model and a Metaheuristic Approach for the Selection and Allocation of Land Parcels Problem

Alejandro Fernández Gil[a,*], Mariam Gómez Sánchez[a], Carlos Castro[a] and Alain Pérez-Alonso[a]

[a]*Departmento de Informática, Universidad Técnica Federico Santa María, 1680, Av. España, Valparaíso, Chile*
E-mail: affernan@jp.inf.utfsm.cl [Fernández Gil]; mggomez@jp.inf.utfsm.cl [Gómez Sánchez]; Carlos.Castro@inf.utfsm.cl [Castro]; alain.perez@usm.cl [Pérez-Alonso]

Received DD MMMM YYYY; received in revised form DD MMMM YYYY; accepted DD MMMM YYYY

Abstract

This article is about the current agricultural scenario, where large-scale production causes large amounts of food to be moved to various points of consumption, causing large emissions of so-called greenhouse gases (GHGs) and increasing the carbon footprint. The land use optimization and the land parcels allocation are important areas of agriculture research that represents one of the greatest current challenges and classifies as combinatorial optimization problems. In this context, the Selection and Allocation of Land Parcels Problem (SA-LPP) is proposed; its goal is to optimize the process of selection and allocation of land parcels in small areas available for food production. We propose a reformulation of the SA-LPP as a variant of the two-dimensional orthogonal packing problem (2OPP), called Group-2OPP. This problem is solved through mixed integer linear programming model, but due to the model complexity , also we propose a GRASP metaheuristic approach. Some sensitivity analyses were also performed to evaluate the impact of parameters on the solutions. Computational results show that the proposed metaheuristic is superior to MILP model in terms of solution quality and computational times.

Keywords: Selection and allocation of land parcels problem; agriculture; combinatorial optimization; mixed integer linear programming; metaheuristic; GRASP; packing problem

1. Introduction

More than ever, agriculture faces new challenges and responsibilities due to new changes taking place at different levels throughout society. Increasing environmental concerns (Assessment et al., 2005) and climate change (Pachauri et al., 2014) make it necessary to adopt innovative and sustainable agricultural

*Author to whom all correspondence should be addressed (e-mail: affernan@jp.inf.utfsm.cl).

techniques to face the challenges of the future (e.g., climate change, overpopulation and depletion of resources) (Cotter et al., 2014).

Large-scale agricultural production causes large logistic operations to move to various points of consumption regardless of the distances traveled, which implies large CO2 emissions along with other greenhouse gases (GHGs) and increased carbon footprint (Yue et al., 2017). The emissions reduction and food risk associated with the transfer is one of the main problems we face today. A practical way to mitigate this problem is based on carrying out food production processes on a small scale and closer to the cities (Wakeland et al., 2012; Adewale et al., 2016), producing small quantities of different varieties of cultivation which ensure fulfill the needs of a specific area and avoid the import and export of food. Currently, the form of small-scale food production can use land parcels or even artificial land dedicated to planting crops, always located as close as possible to the cities. The process of selecting and allocating crops within these lands is a task that must be carried out as efficiently and optimally as possible.

Agricultural Production Planning (APP) is a process of spatial organization of agricultural and forestry products that assigns specific land uses to specific areas (Cardín-Pedrosa and Alvarez-López, 2012). The main objective of this process is to promote sustainable development by optimizing agricultural production systems considering a set of norms or rules related to structural, socioeconomic and environmental conditions (Jones et al., 2017). APP models must be taken into account those products that will be produced, specific areas that will be assigned to each of them (e.g., size, allocation and location), production practices to be used and production costs (Heady, 1954; Shrouf et al., 2014; Osaki and Batalha, 2014).

Many research investigations on this topic demonstrate how Operations Research (OR) methods works in the context of Combinatorial Optimization (CO) area. The optimal or almost optimal solution to these problems is usually obtained through the use of exact or approximate algorithms. Exact algorithms for linear programming problems can only address a relatively small-scale problems (Aerts et al., 2003; Haque and Asami, 2014). The approximate algorithms can find very good solutions close to the optimum for large-scale problems in an affordable time, therefore, are commonly used in practice. Over the past decade a large number of approximate algorithms have been developed. These algorithms includes classic heuristics and metaheuristics to efficiently solve different problems of land optimization use and allocation in agriculture (Memmah et al., 2015; Kaim et al., 2018), for example, the so-called population search algorithms include genetic algorithms (GA) (Liu et al., 2015a,b; Mohammadi et al., 2016; Gliesch et al., 2017), ant colony optimization (ACO) (Yu et al., 2011; Liu et al., 2012; Yang et al., 2012; LIU et al., 2014), artificial bee colony (ABC) (Yang et al., 2015; Wang et al., 2019) and, particle swarm optimization (PSO) (Ma et al., 2011; Liu et al., 2013; Masoomi et al., 2013; Li et al., 2018). Also, there are other very popular trajectory search algorithms, such as tabu search (TS) (Mohammadi et al., 2016; Łampika et al., 2019), simulated annealing (SA) (Santé et al., 2016; Li and Ma, 2018), and greedy algorithms. Greedy algorithms are commonly known in the literature as a technique for solving optimization problems related to APP; e.g., land-use allocation optimization (Porta et al., 2013; Mohammadi et al., 2016; Kennedy et al., 2016; Seo et al., 2018), path-planning (Oksanen and Visala, 2009; Khan et al., 2017; Tong et al., 2017) and, facility-location (Montoya-Torres et al., 2011). Compared to classic heuristics, metaheuristics perform a deeper search of the solution space, allowing lower and sometimes infeasible movements, and the recombination of solutions to build new ones.

As we mentioned earlier, the production of small-scale food for local consumption in nearby areas or within cities represents a sustainable measure that contributes to the emissions' reduction of polluting gases, and production needs to be adjusted to the consumption of each city. To achieve this, the average

consumption of the food must be known, which can be translated into a quantity of plants to be sown and the arrangement of the plants offers different land parcels that can be occupied by one type of crop species. Since the availability of spaces for cultivation in these areas is scarce, the aim of this research is to optimize the process of the selection and allocation of the land parcels. In this sense, from the literature and to the best of our knowledge, the problem previously raised has not yet been investigated.

The contributions of this paper are as follows: (i) we propose a new problem called Selection and Allocation of Land Parcels Problem (SA-LPP); (ii) we carry out an extension of the known two-dimensional orthogonal packing problem (2OPP) consisting in determining if a set of $\mathcal{R}$ of N small rectangles can be packaged in a large rectangle of known width and height (Lodi et al., 2014), which is called Group-2OPP, and we formulate the SA-LPP as a Group-2OPP, this being a generic modeling approach that can be extended to applications in the field of agriculture; (iii) a mixed integer linear programming (MILP) model is presented for solving this problem and implemented in MiniZinc (Stuckey and Tack, 2013) in combination with CPLEX; (iv) due to the large dimension of the problem, we design and implement a greedy random adaptive search procedure (GRASP) (Feo and Resende, 1995; Resende and Ribeiro, 2010; Rubio-Sánchez et al., 2016), which can then be interpreted as a global optimization method and delivers good quality solutions for the problem; (v) we generate a set of instances to validate our approach. In a computational study, we show that our GRASP approach clearly outperforms the implementation performed based on MILP. Moreover, the observed short running times suggests an implementation in a real-time decision support environment.

The remainder of the paper is organized as follows. In Section 2, the SA-LPP is defined and a formulation of SA-LPP as a packing optimization problem is presented. The mathematical formulation for this problem is presented in Section 3. The metaheuristic approach proposed for solving this problem is described in Section 4. Computational experiments and results discussion are reported in Section 5 and 6 respectively. Finally, in Section 7 provides our main conclusions and provide further research lines.

2. Problem description

2.1. The Selection and Allocation of Land Parcels Problem (SA-LPP)

The food transportation process generates large amounts of polluting gases, so it is important to generate food within or near cities in available spaces, to reduce the import and export processes. It is well known that the available spaces within or near cities are scarce and small, due to this, the SA-LPP considers a small-scale production carried out in such a way that the quantity of food to be produced is the necessary for local consumption. In SA-LPP it is assumed that the dimensions of the rectangular area available for cultivation are known. This area depends on the average consumption of each crop species to be planted in a city or area where space is available, this average consumption can be represented as an approximate amount of P plants of a crop species that are necessary for the generation of the average quantity required. Each crop species has its specifications as to the distance that they should be from each other within the row of crops, as well as the separation that there should be between each pair of rows (Gómez et al., 1978), which implies that different arrangements of the plants in the land parcels occur every time in a different area where at least the amount of plants required can be cultivated. So, for a total of P plants to be cultivated we establish a total of $\approx 4\sqrt{P}$ different areas for their cultivation.

For example, if $P = 20$ plants of a certain species, the distance between them within the rows should be around 8 *cm* and the separation between each pair of rows 30 *cm*, obtaining 8 possible dimensions that can be placed vertically or horizontally (90° rotation is considered), generating 16 land parcels dimensions (see Table 1).

Table 1
Feasible dimensions to sow $P = 20$ plants. In the *Distribution* column, the 8 possible distributions for the P plants are shown while in the third and fourth columns, the 16 land parcel dimensions with and without rotation are presented. Note that the separation between each pair of plants and rows is 8 *cm* and 30 *cm* respectively

#Dimension	Distribution				Land parcels dimensions					
					0° rotation			90° rotation		
1	(1	×	20)	⇒	(30	×	160)	(160	×	30)
2	(2	×	10)	⇒	(60	×	80)	(80	×	60)
3	(3	×	7)	⇒	(90	×	56)	(56	×	90)
4	(4	×	5)	⇒	(120	×	40)	(40	×	120)
5	(5	×	4)	⇒	(150	×	32)	(32	×	150)
6	(7	×	3)	⇒	(210	×	24)	(24	×	210)
7	(10	×	2)	⇒	(300	×	16)	(16	×	300)
8	(20	×	1)	⇒	(600	×	8)	(8	×	600)

Figure 1 shows the construction of the land parcels, and 3 of the 16 dimensions (90×56, 120×40 and 150×32) presented in Table 1 were used for this.

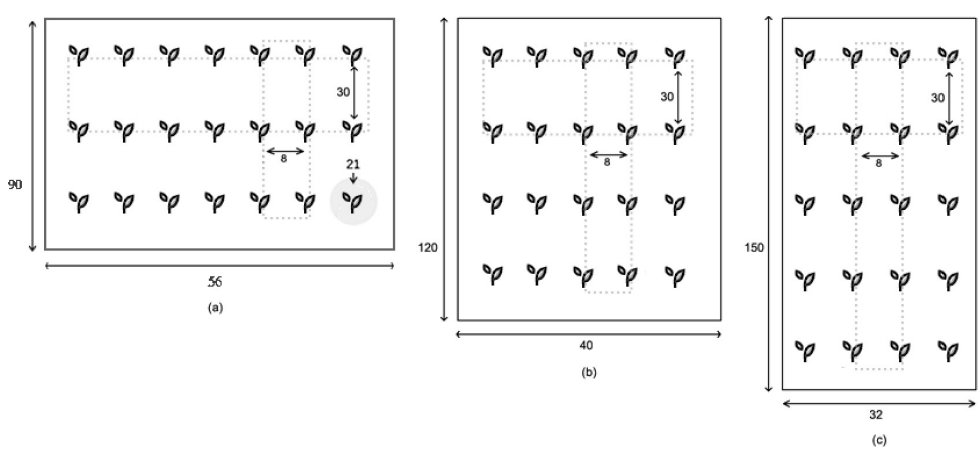

Fig. 1. Arrangement of plants in the land parcels. Note that the parcels have equal or approximate areas, complying with the amount P of crops of a given species.

Afterwards, we have a set of species to be cultivated, and for each crop species we have a set of different possible areas or land parcels that can be occupied for a certain amount of P plants, following the crop specifications mentioned above. Within the area available to cultivate, only one land parcel from each set may be allocated, complying with the following fundamental conditions:

- Land parcels cannot overlap each other.
- Between each pair of land parcels there must be at least a certain distance not cultivated.
- There must be at least a distance between each pair of land parcels and the borders of the area.

As the availability of land to cultivate in urban areas or cities with a growing population is scarce, it may happen that the total species to be cultivated cannot be allocated in the available area, which is why the selection of land parcel sizes should enable the allocation of the largest number of crop species within the available area.

2.2. SA-LPP as a packing optimization problem

To solve the SA-LPP we formulate the problem as a packing optimization problem, being the two-dimensional orthogonal packing problem (2OPP) one of the most similar problems.

The 2OPP is a borderline problem with a two-dimensional bin packing problem (2BPP) and two-dimensional strip packing problem (2DSPP) (Wäscher et al., 2007; Lodi et al., 2014). It is NP-hard as it generalizes the classical one-dimensional bin-packing problem (Johnson et al., 1974; Jansen and Prädel, 2016). In the 2OPP, we seek to determine if a set of rectangles (items) $\mathcal{R} = \{r_1, \ldots, r_N\}$ can be packed into one rectangle of fixed size *Bin*, with fixed width W and height *H* known. Each r_i element has a width w_i and a height h_i with $w_i, h_i \in N$. The elements are not able to overlap with the others and they can be rotated by 90 degree. Each item r_i has a pair of coordinates (x_i, y_i) associated with the bottom left-hand corner (Verstichel et al., 2013).

As an extension of 2OPP we define Group-2OPP. The Group-2OPP considers a set of groups $G = \{g_1, \ldots, g_N\}$, where each g_i represents a set of items $r_{i,j}$ defined by the width and height (small rectangles with dimensions that generate approximate areas, e.g., 1×20, 2×10 and 3×7), and there is a large rectangle called *Bin* of known width *W* and height *H*. This problem consists in placing within the *Bin* at most one item of each g_i, in such a way that between each pair of them there is no overlap and there is at least k *cm* of distance. This distance must also be respected with regard to the borders of the *Bin*. The objective of the problem is to select a set of items, one from each group g_i, so that when packed inside the *Bin* the number of packed items is maximized.

Once presented the definitions in Section 2 where we present SA-LPP and the nomenclature used to describe the Group-2OPP, we have established a correspondence between the two problems (see Table 2. In addition, both problems share basic aspects such as a non-overlapping and objective function. We consider that a preprocessing of the data is necessary to determine the dimensions of each $r_{i,j}$ item, taking into account the distances between the plants and rows established for each type of crop and following the crop marking rules mentioned in the Subsection 2.1. Once this preprocessing is performed, a correct correspondence between the two problems is obtained.

Table 2
Correspondence between the definitions of the SA-LPP and the nomenclature of the Group-2OPP

Group-2OPP		SA-LPP
Bin	=	Area available for cultivation
G	=	Set of species to cultivate
g_i	=	Set of different equivalent areas of land parcel for the crop species i
$r_{i,j}$	=	j-*th* land parcel corresponding to the crop species i
k	=	Distance between land parcels and between each land parcel and the borders of the area available for cultivation

An example of Group-2OPP is shown in Figure 2, using the nomenclature described in Table 2 for the problem and taking k=1. At the top, the set of groups G is shown, and of each g_i their corresponding items $r_{i,j}$. At the bottom, three solutions are shown, for which of each g_i at most one element was selected to be located in the *Bin*. The (Packing 1) is incorrect because the distance between each pair of items and the distance between each item and the borders of the *Bin* is greater than k. The (Packing 2) and (Packing 3) are correct, and (Packing 3) represent the solution that maximizes the number of items packed within the *Bin*.

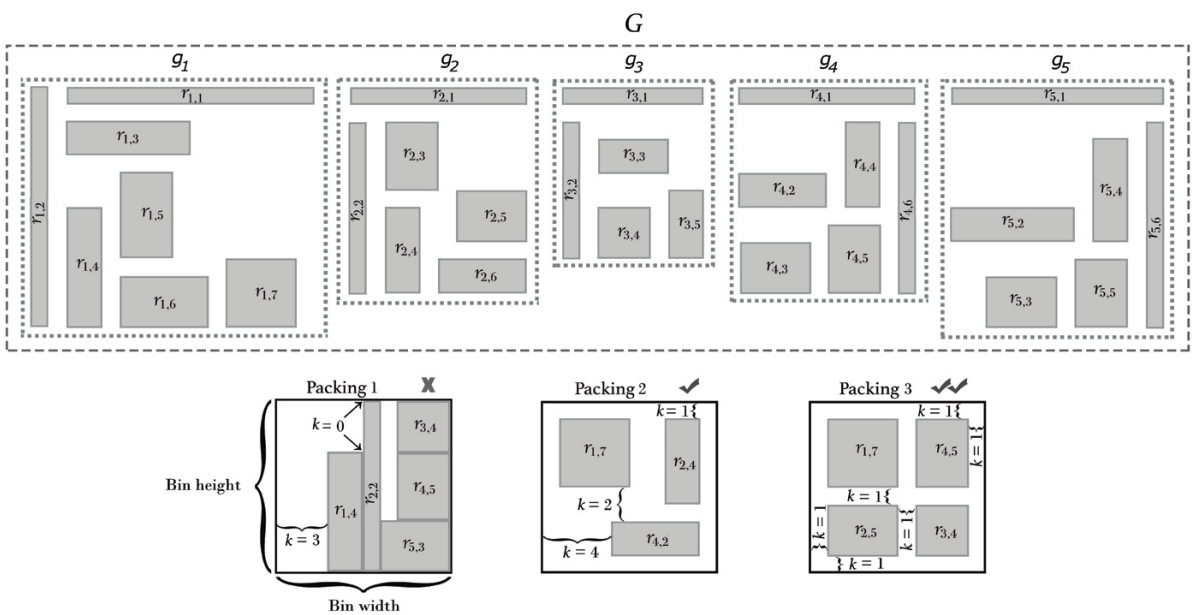

Fig. 2. Visual representation for SA-LPP as a Group-2OPP.

In the following sections, we assume that preprocessing mentioned above to calculate the dimensions of $r_{i,j}$ has been performed and we focus on solving the Group-2OPP.

3. Mixed Integer Linear Programming formulation

In this section we present a MILP for Group-2OPP. Our model has as its objective function (1) the maximization of the quantity of items for packing within the *Bin* (at most one for each group), considering the aspects previously described in Section 2.

Parameters

W : *Bin* width;
H : *Bin* height;
N : Number of groups;
M_x : Upper bound for x;
M_y : Upper bound for y;
C_i : Number of items for group i;
A_{ij} : Width of the item j, of the group i;
L_{ij} : Height of the item j, of the group i;
k : Distance between each pair of items and separation of the borders of the *Bin*.

Decision variables

w_i : Width occupied by group i;
h_i : Height occupied by group i;
x_i : Coordinate x of the lower left corner of the area occupied by group i;
y_i : Coordinate y of the lower left corner of the area occupied by group i;

$$S_{ij} = \begin{cases} 1 & \text{If for the group } i \text{ the item } j \text{ is selected,} \\ 0 & \text{otherwise;} \end{cases}$$

$$I_{il} = \begin{cases} 1 & \text{If the group } i \text{ is on the left of the group } l, \\ 0 & \text{otherwise;} \end{cases}$$

$$J_{il} = \begin{cases} 1 & \text{If the group } i \text{ is on the right of the group } l, \\ 0 & \text{otherwise;} \end{cases}$$

$$D_{il} = \begin{cases} 1 & \text{If the group } i \text{ is before (vertically) the group } l, \\ 0 & \text{otherwise;} \end{cases}$$

$$E_{il} = \begin{cases} 1 & \text{If the group } i \text{ is after (vertically) the group } l. \\ 0 & \text{otherwise.} \end{cases}$$

Objective

$$Max\ \mathcal{Z}_{obj.} = \sum_{i=1}^{N} \sum_{j=1}^{C_i} S_{ij} \tag{1}$$

Constraints

$$\sum_{j=1}^{C_i} S_{ij} \leq 1\ , \forall i = 1 \ldots N \tag{2}$$

$$w_i = \sum_{j=1}^{C_i} A_{ij} \times S_{ij}\ , \forall i = 1 \ldots N \tag{3}$$

$$h_i = \sum_{j=1}^{C_i} L_{ij} \times S_{ij}\ , \forall i = 1 \ldots N \tag{4}$$

$$x_i + w_i + k \leq x_k + (1 - I_{il}) \times M_x\ , \forall i, l = 1 \ldots N, i < l \tag{5}$$
$$x_l + w_l + k \leq x_i + (1 - J_{il}) \times M_x\ , \forall i, l = 1 \ldots N, i < l \tag{6}$$
$$y_i + h_i + k \leq y_k + (1 - D_{il}) \times M_y\ , \forall i, l = 1 \ldots N, i < l \tag{7}$$
$$y_l + h_l + k \leq y_i + (1 - E_{il}) \times M_y\ , \forall i, l = 1 \ldots N, i < l \tag{8}$$

$$x_i + w_i \leq W - k\ , \forall i = 1 \ldots N \tag{9}$$
$$y_i + h_i \leq H - k\ , \forall i = 1 \ldots N \tag{10}$$

$$x_i \geq k\ , \forall i = 1 \ldots N \tag{11}$$
$$y_i \geq k\ , \forall i = 1 \ldots N \tag{12}$$

The formulation uses two types of decision variables, namely continuous and binary. Continuous variables define the dimensions that a certain group of items will occupy within the *Bin*, as well as their respective coordinates. The binary variables establish which item of each group will be selected and also what the position occupied by each group within the *Bin* should be with respect to the remaining groups.

Our formulation contains the constraints defined in the Equations (2)-(12). The Equation (2) states that at most one item can be selected for each group, while Equations (3) and (4) establish that the width and height occupied respectively by each group will be determined by the selected item. The basic constraints of the problem that establish the non-overlap are defined in Equations (5) and (8). The Equations (9) and (10) ensure that the area occupied by each group must be totally within the width and height of the *Bin*, taking into account a separation of at most k *cm* from the edges of the *Bin*, and finally the Equations (11) and (12) establish that the coordinates that define the lower left location point of any item must be greater than or equal to k.

4. Algorithmic solution based on GRASP

The resolution of the problem studied using exhaustive techniques becomes complex, because when adding a group to the problem, this represents the creation of a set of decision variables, so the problem becomes more complex. Based on this, we design and implement a Greedy Randomized Adaptive Search Procedure (GRASP), presented in this section.

GRASP algorithm is a multi-start or iterative method where each iteration has two phases: (i) constructive and (ii) local search. Our metaheuristic approach uses the Equation (1) described in the previous Section 3 as a function of quality. Its general structure is shown in Algorithm 1.

Algorithm 1 GRASP (Iter) Pseudocode

1: $preprocessing()$;
2: $\mathcal{S} = \emptyset$;
3: $i = 0$;
4: **while** $(i < Iter)$ **do**
5: $\quad sol = Construction_Phase(SEL, rcl)$;
6: $\quad sol' = Local_Search(sol, \mathcal{R})$;
7: $\quad updateSolution(sol', \mathcal{S})$;
8: $\quad i = i + 1$;
9: **end while**
10: **return** $\mathcal{S}$;

Initially, data preprocessing is performed, where in addition to what is mentioned in the Section 2, we eliminate in each group those items with a width or height that is greater than the dimensions of the *Bin* in order to ensure that each solution obtained is valid (line 1). Subsequently, (lines 4-9) as many iterations are performed as defined. In each iteration, a constructive algorithm is executed that provides feasible solutions consisting of locating the rectangles inside the *Bin*, in such a way that the constraints of the problem are considered (line 5). A Local Search procedure based on Hill-Climbing First Improvement is applied to the solution obtained (line 6), and then assign in $\mathcal{S}$ the best solution found (line 7). Finally, the algorithm returns the best solution obtained $\mathcal{S}$ in all iterations (line 10). In the following subsections, we describe the components of the GRASP algorithm, which includes the Constructive algorithm and the Local Search algorithm and the methodology for its application to our problem.

4.1. Representation

The first step in implementing our GRASP algorithm is defining the representation of a solution. We define that a solution will be represented as a vector $1 \times N$, where each element r_j contains for each item packaged in the *Bin* the group both the width and height, as well as the position x and y where it was packed.

$$\mathcal{S} = \{\ r_j\ \{(group_j, width_j, height_j, x_j, y_j)\}, \ldots, r_N\{(group_N, width_N, height_N, x_N, y_N)\}\ \}$$

4.2. Construction phase

In this phase, a feasible solution to the problem is built, initially a list called $\mathcal{R}$ is generated, its elements are the rectangles that will be packed in the *Bin*. According to the specifications of the problem, at most one rectangle can be packed per group. In the selection process of the rectangles, we rely on the parameter SEL which would indicate which elements of each group will be taken into account at the time of making the selection, then if SEL takes as values $S1$, $S2$ or $S3$ will only be available in the selection process of the first, second or third section of each group respectively, while if SEL=SC, the selection will be made by taking into account all the elements contained in each group.

The process of generating $\mathcal{R}$ is shown in Algorithm 4.2, where the initialization of $\mathcal{R}$ (line 1) is performed and through the parameter SEL are determined the lower and upper bounds, $\mathcal{L}i$, $\mathcal{L}s$ respectively. Subsequently, each $g_i \in \mathcal{G}$ is traversed and an item is randomly selected taking into consideration the bounds $\mathcal{L}i$, $\mathcal{L}s$ determined (lines 3-5). Once the items are selected, $\mathcal{R}$ is ordered in descending order, according to the maximum height criteria (line 6) and returned (line 7).

Algorithm 2 Generate_List_R(SEL) Pseudocode

1: $\mathcal{R} \leftarrow \emptyset$;
2: $\mathcal{L}i, \mathcal{L}s \leftarrow searchBounds(SEL)$;
3: **for** $g_i \in \mathcal{G}$ **do**
4: $\quad \mathcal{R}.add(g_i[rand(\mathcal{L}i, \mathcal{L}s)])$;
5: **end for**
6: sort $\mathcal{R}$ by maximum height criteria;
7: **return** $\mathcal{R}$;

The Algorithm 3 refers to the constructive procedure implemented. An empty solution (line 1) represents the beginning of the algorithm, and in the (line 2) the set $\mathcal{R}$ is generated (procedure described in Algorithm 4.2). The $Coor$ list will contain the coordinates where it is possible to allocate an element, and it is initialized in the pair (k, k) to respect the separation that must exist between each item and the borders of the *Bin* (line 3).

The selection procedure for the rectangle to be located in the GRASP algorithm is carried out based on a list of candidates that we will call RCL, this list being initialized, at most, with the first rcl elements found in $\mathcal{R}$, from which a candidate will be randomly selected to be placed in the *Bin* (lines 4-6). Then, iteratively while in $\mathcal{R}$ and $Coor$ there are elements (line 7), the existing available space is determined from the first coordinate stored in $Coor$ (line 8); if the available space allows the allocation of the selected candidate, then proceed to the following steps: add the candidate to the solution (line 10); delete the candidate from $\mathcal{R}$ (line 11); calculate and add to $Coor$ the coordinates corresponding to the lower right corner and the upper left corner of the packaged item considering the distance k (line 12 and 13); redefine the RCL candidate list and randomly select an RCL candidate (lines 14-16).

Subsequently, in each iteration, remove from $Coor$ the first position that contains the coordinates where the last candidate was allocated or its allocation was attempted (line 18), and the $Coor$ list is ordered following the first criterion of the minimum coordinate with respect to y, and as a second with respect to x (line 19). Finally, the algorithm returns the built solution (line 21).

Algorithm 3 Construction_Phase(SEL, rcl)

```
solution ← ∅;
R ← Generate_List_R(SEL);
Coor.add((k, k));
Ub ← min(rcl, |R|);
RCL ← R[0 : Ub];
candidate_r ← RCL[rand(0, |RCL| − 1)];
while (R ≠ 0 and Coor ≠ 0) do
    spaceW, spaceH ← DetermineNextSpaceToPack(Coor.first, solution);
    if candidate_r.w ≤ spaceW and candidate_r.h ≤ spaceH; then
        solution.add(candidate_r);
        R.remove(candidate_r);
        coor ← CalculateNewCoordinates();
        Coor.add(coor);
        Ub ← min(rcl, |R|);
        RCL ← R[0 : Ub];
        candidate_r ← RCL[rand(0, |RCL| − 1)];
    end if
    Coor.remove(Coor.first);
    Sort Coor;
end while
return solution;
```

Figure 3 describes the construction process by an example where there are eight groups and therefore starts with eight items within $\mathcal{R}$, and the parameter $rcl = 4$. The following steps detail this process:

- The $Coor$ list is initialized to (k, k) and randomly selected item is at position 0 within the *Bin*.
- The first step of the iterative construction process is to calculate the space available in the *Bin*, in this case, greater than or equal to the dimensions of the selected item, so the allocation of the item can be made, and with this is removed item from $\mathcal{R}$. In addition, the coordinate corresponding to the lower right corner of the previously allocated item is added to $Coor$, and the coordinate corresponding to the upper left corner is not added, because the packaged item occupies all the available space with respect to the height. The first element of $Coor$ (recent allocation coordinate) is deleted, RCL is redefined and a new element is selected to be packaged.
- This procedure does not vary until the box (5), where it is not possible to package the item in the position where it corresponds (coordinate corresponding to the upper left corner of the third packaged item), so this coordinate is removed from $Coor$ and the element is packaged in the next available coordinate (upper left corner corresponding to the second packaged element).
- In the box (6) , the $Coor$ list contains only the coordinate corresponding to the bottom right corner of the last packaged item. The randomly selected item will be the last of RCL and in this coordinate it is not possible to perform the allocation; thus, it is eliminated from $Coor$, leaving this list empty and causing the cycle to stop and then obtaining a constructed solution.

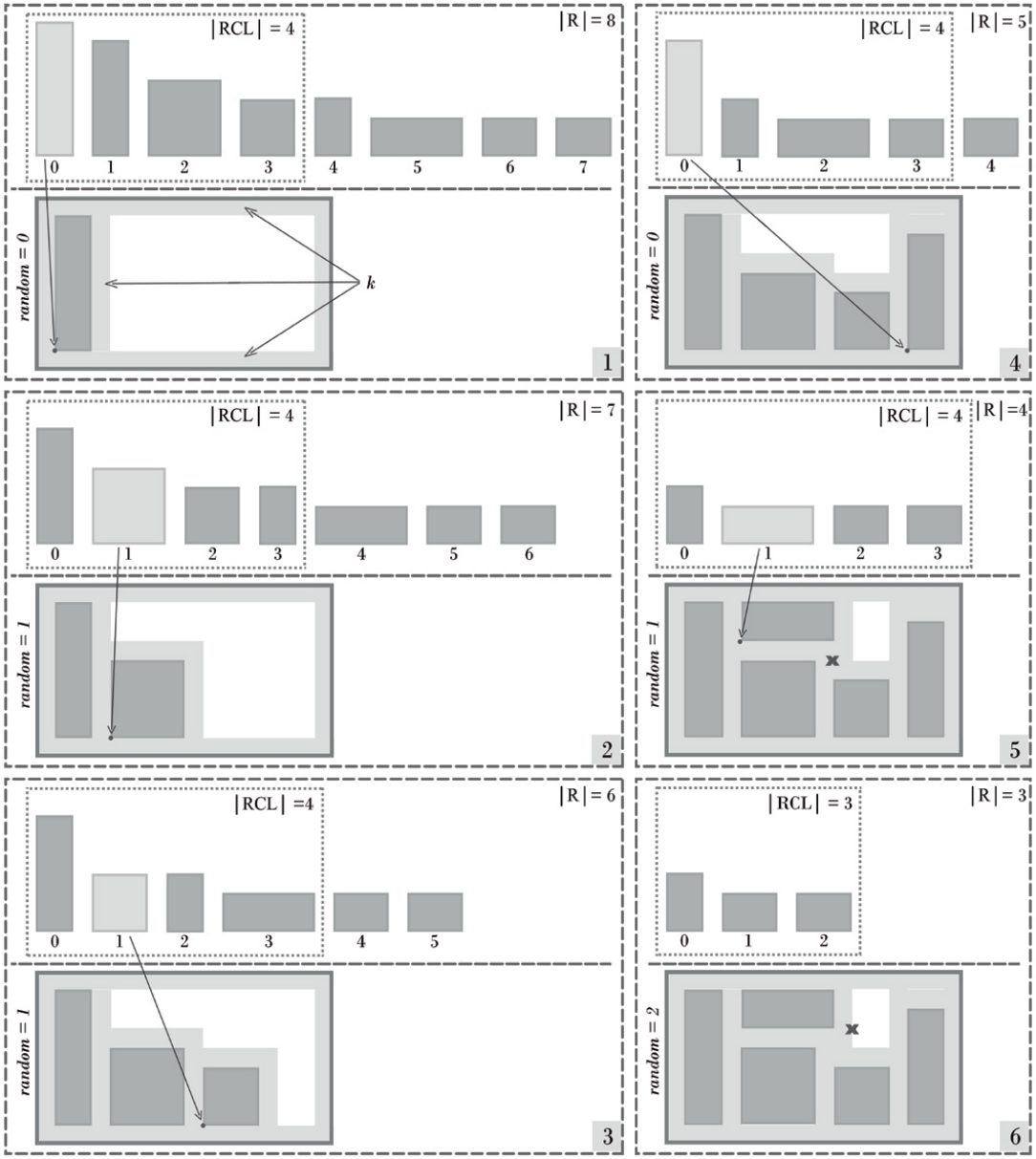

Fig. 3. Allocation method in Constructive phase.

4.3. Local search phase

Local search procedures are iterative algorithms that seek to improve or repair the quality of an initial solution. In our case, when a solution is constructed using the constructive algorithm, unpackaged ele-

ments are stored in $\mathcal{R}$, and it is necessary to locate, if possible, the highest number of these elements in the free spaces within the *Bin*. In this phase, we have defined as neighborhood those solutions where it is possible to pack an item of $\mathcal{R}$ in an empty space inside the *Bin*, performing a procedure based on First Improvement.

The Algorithm 4 shows the Local Search algorithm, this algorithm receives the solution obtained in the constructive algorithm and the list of items $\mathcal{R}$. Initially, the set of empty spaces within the solution (line 2) is determined. Iteratively as long as there are elements in $\mathcal{R}$ and available spaces (line 3), we select randomly an item of $\mathcal{R}$ (line 4) and the available spaces are traveled to try to allocate the item (line 5); If it was possible to allocate the item, then it is added to the solution (lines 7 and 8) and the available spaces are updated (line 9). Once the available spaces have been checked or an item is allocated, the corresponding item (whether allocated or not) is removed from $\mathcal{R}$ (line 14). Finally, the algorithm returns the solution found (line 16).

Algorithm 4 Local_Search (sol, $\mathcal{R}$)

1: $spaces \leftarrow getSpaces(sol)$;
2: **while** ($\mathcal{R} \neq 0$ **and** $spaces \neq 0$) **do**
3: $SA \leftarrow \mathcal{R}[rand(0, min(4, |R|))]$;
4: **for** $space \in spaces$ **do**
5: **if** $SA.width \leq space.width$ **and** $SA.height \leq space.height$ **then**
6: $rep \leftarrow (SA.group, SA.width, SA.height, space.x, space.y)$;
7: $sol.add(space.index, rep)$;
8: $spaces \leftarrow getSpaces(sol)$;
9: **break**;
10: **end if**
11: **end for**
12: $\mathcal{R}.remove(SA)$;
13: **end while**
14: **return** sol;

Following as a reference the example explained in the construction phase, Figure 4 shows the Local Search procedure. In block (1) empty spaces in the *Bin* are shown in dotted points (for this example only one space). In this case $\mathcal{R}$ is not empty and there are some empty spaces, so block (2) shows the random selection of an item of $\mathcal{R}$ which cannot be inserted because the width is greater than available, as there is only one empty space, the element must be removed from $\mathcal{R}$. In block (3), the random selection of an item that can be inserted in the existing space is shown, which completes the empty spaces in the *Bin* and finishes the algorithm because despite the fact that there are elements in $\mathcal{R}$ there are no spaces available inside the *Bin*.

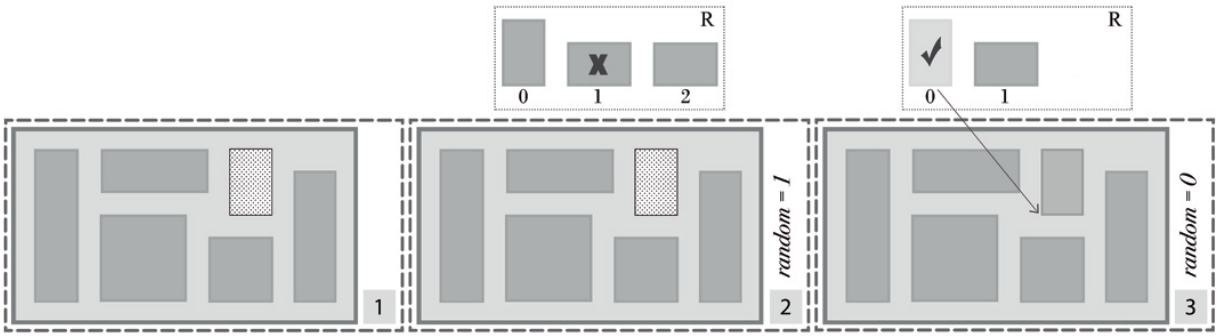

Fig. 4. Allocation method in Local search phase.

5. Computational experiments

This section is devoted to the assessment of the performance of our mixed integer programming model and the metaheuristic approach. We solve the resulting SA-LPP using MiniZinc v2.3.2 in combination with CPLEX v12.9.0 and GRASP approach in C++11 on an Intel(R) Xeon(R) CPU E3-1220 3.10 GHz with 32GB RAM memory, running on the CentOS Linux 7.6.18.19 operating system. We considered a set of randomly generated problem instances, and we performed a number of 100 iterations and 100 trials for each problem instance.

5.1. Problem Instances

In order to evaluate the performance of the model and the metaheuristic proposed in this work, a set of randomly generated instances is proposed for the SA-LPP. Nine classes of instances were defined and three instances were generated within each class. The distance between plants and between rows in a land parcel is generated for each intance in a range of 30 cm to 100 cm. Table 3 shows the parameters of the generated instances, column 1 indicates the number of species to cultivate that each instance includes within each classification, columns 2 and 3 indicate the ranges in which the values of the number of plants for each group and the dimensions of width and height of the area available for cultivation, respectively. Column 4 shows the identifier of the generated instance and columns 5, 6 and 7 show the width and height values of the area available for cultivation, and generated k value for the corresponding instance respectively. The displayed dimensions are in cm.

5.2. Parameter analysis in the GRASP algorithm

Taking into account that each g_i is organized in descending order by height, the value assigned to the parameter SEL could generate differences or similarities, as appropriate, in the composition of $\mathcal{R}$. To define the value of SEL parameter, we perform experiments with all possible options. We also define the

rcl parameter which determines the length of the RCL list. The RCL list is constructed from $\mathcal{R}$ and this list is always sorted in descending order by the maximum height criteria, so this parameter can impact significantly on the difference of the items selected consecutively with respect to their height. To define the restricted candidate list, we perform experiments with the values $rcl \in \{3, 5, 7\}$.

Table 3
Description of the generated instances.

Number of species	Number of plants of each specie	Dimensions of the area (Width and Height)	Identifier of instance	Area Width	Area Height	k
10	$30-35$	$2000-2500$	rnd10_01	2060	2207	87
			rnd10_02	2179	2025	90
			rnd10_03	2233	2004	87
15	$36-40$	$2500-3000$	rnd15_01	2694	2638	57
			rnd15_02	2543	2513	90
			rnd15_03	2989	2504	91
20	$41-45$	$3000-3500$	rnd20_01	3455	3367	76
			rnd20_02	3033	3449	78
			rnd20_03	3214	3029	74
25	$46-50$	$3500-4000$	rnd25_01	3791	3697	63
			rnd25_02	3926	3971	79
			rnd25_03	3914	3681	66
30	$51-55$	$4000-4500$	rnd30_01	4389	4329	80
			rnd30_02	4257	4324	97
			rnd30_03	4123	4317	71
35	$56-60$	$4500-5000$	rnd35_01	4795	4955	81
			rnd35_02	4539	4689	50
			rnd35_03	4583	4818	66
40	$61-65$	$5000-5500$	rnd40_01	5415	5442	75
			rnd40_02	5098	5460	90
			rnd40_03	5447	5275	75
45	$66-70$	$5500-6000$	rnd45_01	5508	5616	85
			rnd45_02	5563	5747	55
			rnd45_03	5539	5821	92
50	$71-75$	$6000-6500$	rnd50_01	6420	6283	58
			rnd50_02	6015	6047	84
			rnd50_03	6074	6398	96

Figures 5, 6 and 7 show the boxplots of the performance obtained by our approach in 100 trials for the first instance generated of each category (by number of groups). To each pair (Instance-SEL) is showed of three boxplots corresponding to a value of rcl.

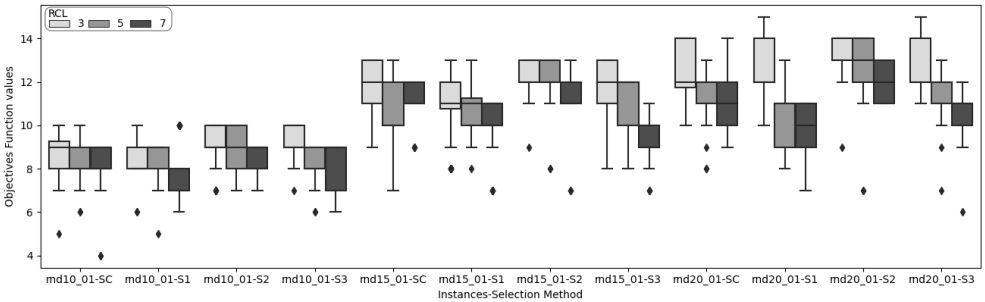

Fig. 5. Boxplot showing the objective function values to solutions found for the instances rnd10_01, rnd15_01 and rnd20_01 and our approach according to the rcl and SEL.

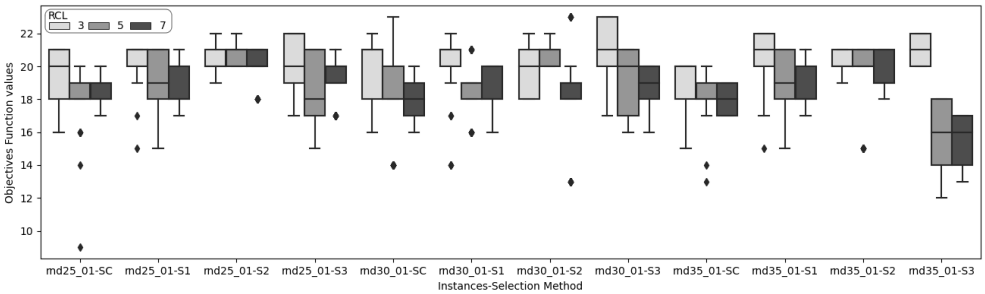

Fig. 6. Boxplot showing the objective function values to solutions found for the instances rnd25_01, rnd30_01 and rnd35_01 and our approach according to the rcl and SEL.

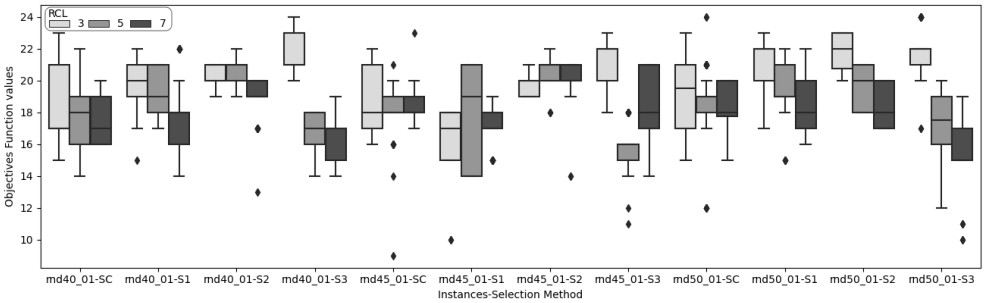

Fig. 7. Boxplot showing the objective function values to solutions found for the instances rnd40_01, rnd45_01 and rnd50_01 and our approach according to the rcl and SEL.

The stability of the algorithm is affected as the complexity of the instances increases; despite this,

in most cases the best solutions were not found as outliers. The most stable behavior of the algorithm occurs when SEL=$S2$, this is due to the constructive process of $\mathcal{R}$ since when SEL=$S2$, those dimensions are selected where the ratio between width and height is closer to 1, and the allocation process is similar in each case, but when SEL takes another value, the ratio between width and height of the elements that are selected from the groups during the $\mathcal{R}$ build process can be very away from 1, so $\mathcal{R}$ will contain dimensions that can be very different and increasingly complexity of the allocation process.

In order to find the parameter configuration of our GRASP algorithm that provides the best results, we apply the algorithm to the 27 proposed instances for each possible (Instance-SEL-rcl). The Table 4 summarizes the values of the best solutions obtained in each case.

Table 4
Greedy Randomized Adaptive Search Procedure: values of the objective values of the best solution out of 100 trials

	SEL=SC			SEL=$S1$			SEL=$S2$			SEL=$S3$		
Instance	rcl_3	rcl_5	rcl_7	rcl_3	rcl_5	rcl_7	rcl_3	rcl_5	rcl_7	rcl_3	rcl_5	rcl_7
rnd10_01	10	10	9	10	9	10	10	10	9	10	9	9
rnd10_02	10	10	9	9	9	8	10	9	9	10	9	8
rnd10_03	9	8	8	9	8	9	9	9	9	10	10	8
rnd15_01	13	13	12	13	13	11	13	13	13	13	12	11
rnd15_02	12	12	11	11	12	13	13	13	12	13	12	13
rnd15_03	13	13	12	13	12	12	13	12	11	13	12	11
rnd20_01	14	13	14	15	13	11	14	14	13	15	13	12
rnd20_02	13	13	12	14	13	12	14	13	13	13	13	12
rnd20_03	10	10	10	11	10	10	11	10	10	11	10	10
rnd25_01	21	20	20	21	21	21	22	22	21	22	21	21
rnd25_02	19	19	20	21	20	19	21	20	20	21	19	19
rnd25_03	15	14	13	14	13	13	14	13	13	16	14	14
rnd30_01	22	23	20	22	21	20	22	22	23	23	21	20
rnd30_02	20	20	18	21	18	18	20	20	18	20	20	20
rnd30_03	20	17	19	20	19	18	20	19	19	21	19	21
rnd35_01	20	20	19	22	21	21	21	21	21	22	18	17
rnd35_02	17	17	17	16	17	17	16	16	16	17	16	16
rnd35_03	18	19	17	19	18	18	17	15	15	17	15	17
rnd40_01	23	22	20	22	21	22	21	22	20	24	18	19
rnd40_02	22	21	22	22	21	21	21	21	21	21	21	21
rnd40_03	20	19	20	20	19	19	20	20	21	21	20	20
rnd45_01	22	21	23	18	21	19	21	22	21	23	18	21
rnd45_02	22	19	21	21	18	22	23	21	21	20	22	19
rnd45_03	19	19	20	21	21	20	19	19	19	21	20	10
rnd50_01	23	24	20	23	22	22	23	21	20	24	20	19
rnd50_02	20	19	19	19	19	19	19	19	19	20	19	19
rnd50_03	23	22	22	23	23	23	23	22	23	22	22	22

Based on the previous results, we used the Friedman-k Related Samples test to assess the importance of our results. Our null hypothesis is that there are no significant differences between the distributions of

each triad (Instance-SEL-rcl), while the alternative hypothesis is based on whether there are significant differences between them. Table 5 summarizes the results obtained.

Because the value corresponding to $\mathcal{X}^2$ for 11 degrees of freedom and a significance level of $\alpha = 0.05$ is 19.675 and our test statistic is 81.673, the null hypothesis is rejected, so if there are significant differences in the distributions of each triad (Instance-SEL-rcl).

Table 5
Statistical comparision between each pair (SEL-rcl) using Friedman-k Related Samples test

Friedman Test	
Total N	27.0
Contrast statistic	81.673
Degrees of freedom	11.0
Asymp. Sig. (bilateral test)	0.00

Table 6 shows the average ranges obtained in the Friedman-k Related Samples test for each combination (SEL-rcl). The results show that for each value of SEL, the best performing value of rcl is 3 and the combination (SEL=3, rcl=3) provides the best results, that is why, we will use this combination of parameters for the analysis and comparison of the results.

Table 6
Average ranges obtained in the Friedman-k Related Samples test

	$SEL = SC$			$SEL = S1$			$SEL = S2$			$SEL = S3$		
	rcl_3	rcl_5	rcl_7	rcl_3	rcl_5	rcl_7	rcl_3	rcl_5	rcl_7	rcl_3	rcl_5	rcl_7
Range	8.19	6.24	5.02	8.35	5.67	5.39	8.20	6.52	5.57	9.69	5.06	4.11

6. Results analysis and comparisons

Table 7 summarizes the computational results provided by MiniZinc in combination with CPLEX from our MILP model (MILP column) in terms of objective function ($\mathcal{Z}_{obj.}$) and computational time (Time(s)). For the GRASP approach (GRASP column), the best objective function ($\mathcal{G}_{obj.}$) and execution time (Time(s)) are presented. The values of Gap among the solutions obtained are reported. The results presented have been calculated following the parameter configuration described above, for the 27 proposed instances, and a maximum execution time of 2 hours. For the proposed MILP model, if the optimal solution for a given instance cannot be found due to the default execution time, then the best value found is reported followed by α.

Table 7
Solutions obtained by our MILP model and GRASP approach

Problem					Our approaches				
Group-2OPP					MILP		GRASP		
Intance	$\|G\|$	W	H	k	$\mathcal{Z}_{obj.}$	Time (s)	$\mathcal{G}_{obj.}$	Time (s)	Gap (%)
rnd10_01	10	2060	2207	87	10	2350.61	10	7.28	0.00
rnd10_02	10	2179	2025	90	10	1938.15	10	7.89	0.00
rnd10_03	10	2233	2004	87	8^{α}	7200.00	10	5.51	20.00
rnd15_01	15	2694	2638	57	10^{α}	7200.00	13	14.23	23.08
rnd15_02	15	2543	2513	90	12^{α}	7200.00	13	16.73	7.69
rnd15_03	15	2989	2504	91	12^{α}	7200.00	13	15.49	7.69
rnd20_01	20	3455	3367	76	12^{α}	7200.00	15	18.98	20.00
rnd20_02	20	3033	3449	78	11^{α}	7200.00	13	25.45	15.38
rnd20_03	20	3214	3029	74	10^{α}	7200.00	11	20.38	9.09
rnd25_01	25	3791	3697	63	18^{α}	7200.00	22	30.45	18.18
rnd25_02	25	3926	3971	79	19^{α}	7200.00	21	43.89	9.52
rnd25_03	25	3914	3681	66	13^{α}	7200.00	16	32.74	18.75
rnd30_01	30	4389	4329	80	17^{α}	7200.00	23	45.23	26.09
rnd30_02	30	4257	4324	97	17^{α}	7200.00	20	48.54	15.00
rnd30_03	30	4123	4317	71	17^{α}	7200.00	21	44.55	19.05
rnd35_01	35	4795	4955	81	17^{α}	7200.00	22	60.24	22.73
rnd35_02	35	4539	4689	50	16^{α}	7200.00	17	63.72	5.88
rnd35_03	35	4583	4818	66	12^{α}	7200.00	17	61.52	29.41
rnd40_01	40	5415	5442	75	15^{α}	7200.00	24	85.03	37.50
rnd40_02	40	5098	5460	90	21^{α}	7200.00	21	89.42	0.00
rnd40_03	40	5447	5275	75	19^{α}	7200.00	21	98.53	9.52
rnd45_01	45	5508	5616	85	15^{α}	7200.00	23	108.13	34.78
rnd45_02	45	5563	5747	55	17^{α}	7200.00	20	112.63	15.00
rnd45_03	45	5539	5821	92	19^{α}	7200.00	21	120.27	9.52
rnd50_01	50	6420	6283	58	21^{α}	7200.00	24	140.53	12.50
rnd50_02	50	6015	6047	84	19^{α}	7200.00	20	137.71	5.00
rnd50_03	50	6074	6398	96	22^{α}	7200.00	22	143.01	0.00

The results obtained by our GRASP algorithm show that for the Small instances the optimum value is found by both methods, while for the Medium and Large instances, good quality solutions are achieved and in low cost computational times.

Our metaheuristic algorithm surpasses our MILP model, starting with the rnd10_03 instance. We use a maximum execution time of 2 hours, where the MILP model does not get results, but if it provides lower bounds of the solutions, this happens because the problem has a rapid scalability. In the column (Gap(%)) the gap between both solutions is shown ($100 \times (\mathcal{G}_{obj.} - \mathcal{Z}_{obj.})/\mathcal{G}_{obj.}$), selecting as the best solution obtained those presented in column $\mathcal{G}_{obj.}$, demonstrating that in the indicated times the approximate GRASP approach surpasses the implementation of the proposed model.

In order to analyze the contribution of each phase of the GRASP algorithm in the Table 8, we report the results obtained by the algorithm taking into account each phase individually. In the column *Construction*, we have the values obtained by our construction phase; then, in *Local Search* we present the

solution obtained including the Local Search phase; these results correspond to the best solution obtained. The *AV Construction* and *AV Local Search* columns show the averages obtained from the total of solutions found by the algorithm for the constructive and local search phase, respectively.

Table 8
Comparison of component performance for set of instances

Instance	Construction	Local Search	AV Construction	AV Local Search
rnd10_01	10	10	9.10	9.22
rnd10_02	8	10	7.00	9.00
rnd10_03	10	10	7.72	8.86
rnd15_01	11	13	10.53	12.07
rnd15_02	12	13	9.81	11.96
rnd15_03	13	13	12.20	12.67
rnd20_01	14	15	11.79	13.00
rnd20_02	9	13	7.66	11.91
rnd20_03	8	11	6.50	10.01
rnd25_01	18	22	16.00	20.08
rnd25_02	20	21	18.38	20.92
rnd25_03	12	16	10.11	15.00
rnd30_01	21	23	18.03	21.07
rnd30_02	18	20	16.01	19.02
rnd30_03	21	21	17.12	20.03
rnd35_01	22	22	19.50	20.89
rnd35_02	17	17	15.00	15.98
rnd35_03	13	17	10.59	16.00
rnd40_01	22	24	20.83	21.65
rnd40_02	17	21	15.26	19.83
rnd40_03	16	21	13.97	19.38
rnd45_01	19	23	17.00	20.99
rnd45_02	18	20	17.10	18.00
rnd45_03	20	21	16.93	20.00
rnd50_01	17	24	16.01	21.80
rnd50_02	18	20	17.22	18.47
rnd50_03	19	22	17.41	19.07

These results demonstrate the importance of Local Search phase for our algorithm because in most instances the Local Search represents a significant improvement with respect to the quality of the solution obtained in the Construction phase, which can also be seen with respect to the averages presented of both phases.

7. Conclusions and future work

In this work, we have presented the Selection and Allocation of Land Parcels Problem (SA-LPP) aimed to optimizing nearby rectangular areas or within the cities available for food production. At the light of the analysis and results obtained, we found that solving this problem that promotes small-scale food production can help reduce the carbon footprint and GHG emissions that are produced by moving food.

In order to formulate this problem we made an extension of the known packing 2OPP called Group-2OPP, with which we cover all the constraints of the SA-LPP. For solving this problem we have proposed a MILP mathematical formulation. Furthermore, since the formulation is not able to provide a good quality solution within reasonable time limits for some instances, a Greedy Randomized Adaptive Search Procedure (GRASP) metaheuristic algorithm is proposed, stochastic and greedy step-wise construction procedure. In order to asses their performance, a set of instances have been generated randomly and classified according to their difficulty of resolution.

From the computational experiments carried out in this work, it can be concluded that the problem is difficult to solve until optimality, especially in cases of medium and large problems. However, the GRASP approach proposed in this work are able to provide good-quality solutions in short computational times. The use of exact techniques, as can be seen from the computational resulting using MiniZinc in combination with CPLEX, requires a high computational effort in terms of execution time especially in the Medium and Large instances, which would cause delay or slow-down in the decision-making process during the optimization of the land parcels assignment to a given area. Whereas, the computational effort required by this GRASP algorithm is not strongly influenced by the difficulty of the instances, since it provides feasible solutions with a similar computational effort for instances of different sizes. Hence, the use of approximate techniques are appropriate for quickly getting a feasible optimization in the allocation of land parcels. This feature makes the use of GRASP suitable for cases where the number and quantity of species to be produced may vary.

Considering the contributions presented in this paper, the next stage of our research will be focused on the analysis of requirements of each type of species such as land parcel properties, irrigation time, production cycle, etc. In addition, constraints that address adjacency or non-adjacency between certain types of crops could be considered. Finally, another open line for future research is to look at the problem studied in a three-dimensional mode, promoting greater use of the space available for food production.

Acknowledgments

This work has been partially supported by ANID-PFCHA/Doctorado Nacional /2020-21200871 and CONICYT-PFCHA (Doctorado Nacional /2017-21171857), and in part by Proyectos de Línea de Investigación Regular (PI_LIR_2020_67, UTFSM) and Programa de Incentivo a la Iniciación Científica (PIIC, UTFSM). We are grateful to anonymous referees, whose valuable comments helped us to strongly improve the paper.

References

Adewale, C., Higgins, S., Granatstein, D., Stöckle, C.O., Carlson, B.R., Zaher, U.E., Carpenter-Boggs, L., 2016. Identifying hotspots in the carbon footprint of a small scale organic vegetable farm. *Agricultural Systems* 149, 112 – 121.

Aerts, J.C.J.H., Eisinger, E., Heuvelink, G.B.M., Stewart, T.J., 2003. Using linear integer programming for multi-site land-use allocation. *Geographical Analysis* 35, 2, 148–169.

Assessment, M.E., et al., 2005. *Ecosystems and human well-being*, Vol. 5. Island press Washington, DC:.

Cardín-Pedrosa, M., Alvarez-López, C.J., 2012. Reprint of: Model for decision-making in agricultural production planning. *Computers and Electronics in Agriculture* 86, 131 – 139. Multi-scale water and land-use modelling for better decision making in agro-eco systems.

Cotter, M., Berkhoff, K., Gibreel, T., Ghorbani, A., Golbon, R., Nuppenau, E.A., Sauerborn, J., 2014. Designing a sustainable land use scenario based on a combination of ecological assessments and economic optimization. *Ecological Indicators* 36, 779 – 787.

Feo, T.A., Resende, M.G.C., 1995. Greedy randomized adaptive search procedures. *Journal of Global Optimization* 6, 2, 109–133.

Gliesch, A., Ritt, M., Moreira, M.C.O., 2017. A genetic algorithm for fair land allocation. In *Proceedings of the Genetic and Evolutionary Computation Conference*, ACM, Association for Computing Machinery, New York, NY, USA, p. 793–800.

Gómez, A., Marco, O., de Extensión Agraria, E.S., 1978. *Marqueo de plantaciones*. Hojas divulgadoras. Ministerio de Agricultura. Publicaciones de Extensión Agraria.

Haque, A., Asami, Y., 2014. Optimizing urban land use allocation for planners and real estate developers. *Computers, Environment and Urban Systems* 46, 57 – 69.

Heady, E.O., 1954. Simplified presentation and logical aspects of linear programming technique. *Journal of Farm Economics* 36, 5, 1035–1048.

Jansen, K., Prädel, L., 2016. New approximability results for two-dimensional bin packing. *Algorithmica* 74, 1, 208–269.

Johnson, D.S., Demers, A., Ullman, J.D., Garey, M.R., Graham, R.L., 1974. Worst-case performance bounds for simple one-dimensional packing algorithms. *SIAM Journal on Computing* 3, 4, 299–325.

Jones, J.W., Antle, J.M., Basso, B., Boote, K.J., Conant, R.T., Foster, I., Godfray, H.C.J., Herrero, M., Howitt, R.E., Janssen, S., Keating, B.A., Munoz-Carpena, R., Porter, C.H., Rosenzweig, C., Wheeler, T.R., 2017. Toward a new generation of agricultural system data, models, and knowledge products: State of agricultural systems science. *Agricultural Systems* 155, 269 – 288.

Kaim, A., Cord, A.F., Volk, M., 2018. A review of multi-criteria optimization techniques for agricultural land use allocation. *Environmental Modelling & Software* 105, 79 – 93.

Kennedy, C.M., Hawthorne, P.L., Miteva, D.A., Baumgarten, L., Sochi, K., Matsumoto, M., Evans, J.S., Polasky, S., Hamel, P., Vieira, E.M., Develey, P.F., Sekercioglu, C.H., Davidson, A.D., Uhlhorn, E.M., Kiesecker, J., 2016. Optimizing land use decision-making to sustain brazilian agricultural profits, biodiversity and ecosystem services. *Biological Conservation* 204, 221 – 230.

Khan, A., Noreen, I., Habib, Z., 2017. On complete coverage path planning algorithms for non-holonomic mobile robots: Survey and challenges. *J. Inf. Sci. Eng.* 33, 101–121.

Łampika, Ł., Musiał, K., Burduk, A., 2019. Case study of production planning optimization with use of the greedy and tabu search algorithms. In Burduk, A., Chlebus, E., Nowakowski, T. and Tubis, A. (eds), *Intelligent Systems in Production Engineering and Maintenance*, Springer, Springer International Publishing, Cham, pp. 66–75.

Li, F., Gong, Y., Cai, L., Sun, C., Chen, Y., Liu, Y., Jiang, P., 2018. Sustainable land-use allocation: A multiobjective particle swarm optimization model and application in changzhou, china. *Journal of Urban Planning and Development* 144, 2, 04018010.

Li, X., Ma, X., 2018. An improved simulated annealing algorithm for interactive multi-objective land resource spatial allocation. *Ecological Complexity* 36, 184 – 195.

Liu, X., Li, X., Shi, X., Huang, K., Liu, Y., 2012. A multi-type ant colony optimization (maco) method for optimal land use allocation in large areas. *International Journal of Geographical Information Science* 26, 7, 1325–1343.

Liu, X., Ou, J., Li, X., Ai, B., 2013. Combining system dynamics and hybrid particle swarm optimization for land use allocation. *Ecological Modelling* 257, 11 – 24.

LIU, Y., TANG, D., KONG, X., LIU, Y., AI, T., 2014. A land-use spatial allocation model based on modified ant colony

optimization. *INTERNATIONAL JOURNAL OF ENVIRONMENTAL RESEARCH (IJER)* 8, 4, 1115–1126. Export Date: Jul 14 2020 12:00AM.

Liu, Y., Tang, W., He, J., Liu, Y., Ai, T., Liu, D., 2015a. A land-use spatial optimization model based on genetic optimization and game theory. *Computers, Environment and Urban Systems* 49, 1 – 14.

Liu, Y., Yuan, M., He, J., Liu, Y., 2015b. Regional land-use allocation with a spatially explicit genetic algorithm. *Landscape and Ecological Engineering* 11, 1, 209–219.

Lodi, A., Martello, S., Monaci, M., Vigo, D., 2014. Two-Dimensional Bin Packing Problems, John Wiley & Sons, Ltd, chapter 5. pp. 107–129.

Ma, S., He, J., Liu, F., Yu, Y., 2011. Land-use spatial optimization based on pso algorithm. *Geo-spatial Information Science* 14, 1, 54–61.

Masoomi, Z., Mesgari, M.S., Hamrah, M., 2013. Allocation of urban land uses by multi-objective particle swarm optimization algorithm. *International Journal of Geographical Information Science* 27, 3, 542–566.

Memmah, M.M., Lescourret, F., Yao, X., Lavigne, C., 2015. Metaheuristics for agricultural land use optimization. a review. *Agronomy for Sustainable Development* 35, 3, 975–998.

Mohammadi, M., Nastaran, M., Sahebgharani, A., 2016. Development, application, and comparison of hybrid meta-heuristics for urban land-use allocation optimization: Tabu search, genetic, grasp, and simulated annealing algorithms. *Computers, Environment and Urban Systems* 60, 23 – 36.

Montoya-Torres, J.R., Aponte, A., Rosas, P., 2011. Applying grasp to solve the multi-item three-echelon uncapacitated facility location problem. *Journal of the Operational Research Society* 62, 2, 397–406.

Oksanen, T., Visala, A., 2009. Coverage path planning algorithms for agricultural field machines. *Journal of Field Robotics* 26, 8, 651–668.

Osaki, M., Batalha, M.O., 2014. Optimization model of agricultural production system in grain farms under risk, in sorriso, brazil. *Agricultural Systems* 127, 178 – 188.

Pachauri, R.K., Allen, M.R., Barros, V.R., Broome, J., Cramer, W., Christ, R., Church, J.A., Clarke, L., Dahe, Q., Dasgupta, P., Dubash, N.K., Edenhofer, O., Elgizouli, I., Field, C.B., Forster, P., Friedlingstein, P., Fuglestvedt, J., Gomez-Echeverri, L., Hallegatte, S., Hegerl, G., Howden, M., Jiang, K., Cisneroz, B.J., Kattsov, V., Lee, H., Mach, K.J., Marotzke, J., Mastrandrea, M.D., Meyer, L., Minx, J., Mulugetta, Y., O'Brien, K., Oppenheimer, M., Pereira, J.J., Pichs-Madruga, R., Plattner, G.K., Pörtner, H.O., Power, S.B., Preston, B., Ravindranath, N.H., Reisinger, A., Riahi, K., Rusticucci, M., Scholes, R., Seyboth, K., Sokona, Y., Stavins, R., Stocker, T.F., Tschakert, P., van Vuuren, D., van Ypserle, J.P., 2014. *Climate Change 2014: Synthesis Report. Contribution of Working Groups I, II and III to the Fifth Assessment Report of the Intergovernmental Panel on Climate Change*. IPCC, Geneva, Switzerland.

Porta, J., Parapar, J., Doallo, R., Barbosa, V., Santé, I., Crecente, R., Díaz, C., 2013. A population-based iterated greedy algorithm for the delimitation and zoning of rural settlements. *Computers, Environment and Urban Systems* 39, 12 – 26.

Resende, M.G., Ribeiro, C.C., 2010. Greedy Randomized Adaptive Search Procedures: Advances, Hybridizations, and Applications, Springer US, Boston, MA. pp. 283–319.

Rubio-Sánchez, M., Gallego, M., Gortázar, F., Duarte, A., 2016. Grasp with path relinking for the single row facility layout problem. *Knowledge-Based Systems* 106, 1 – 13.

Santé, I., Rivera, F.F., Crecente, R., Boullón, M., Suárez, M., Porta, J., Parapar, J., Doallo, R., 2016. A simulated annealing algorithm for zoning in planning using parallel computing. *Computers, Environment and Urban Systems* 59, 95 – 106.

Seo, M., Shin, H., Tsourdos, A., 2018. Task allocation in agricultural remote sensing applications using submodular maximization algorithm. In *2018 37th Chinese Control Conference (CCC)*, IEEE, pp. 6860–6865.

Shrouf, F., Ordieres-Meré, J., García-Sánchez, A., Ortega-Mier, M., 2014. Optimizing the production scheduling of a single machine to minimize total energy consumption costs. *Journal of Cleaner Production* 67, 197 – 207.

Stuckey, P.J., Tack, G., 2013. Minizinc with functions. In Gomes, C. and Sellmann, M. (eds), *Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems*, Springer, Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 268–283.

Tong, J., Wu, C., Jiang, H., Yu, Y., Rao, X., 2017. Optimizing the path of seedling low-density transplanting by using greedy genetic algorithm. *Computers and Electronics in Agriculture* 142, 356 – 368.

Verstichel, J., De Causmaecker, P., Berghe, G.V., 2013. An improved best-fit heuristic for the orthogonal strip packing problem. *International Transactions in Operational Research* 20, 5, 711–730.

Wakeland, W., Cholette, S., Venkat, K., 2012. Food transportation issues and reducing carbon footprint, Springer US, Boston,

MA. pp. 211–236.

Wang, J., Wang, Z., Li, X., Zhou, H., 2019. Artificial bee colony-based combination approach to forecasting agricultural commodity prices. *International Journal of Forecasting*

Wäscher, G., Haußner, H., Schumann, H., 2007. An improved typology of cutting and packing problems. *European Journal of Operational Research* 183, 3, 1109 – 1130.

Yang, L., Sun, X., Peng, L., Shao, J., Chi, T., 2015. An improved artificial bee colony algorithm for optimal land-use allocation. *International Journal of Geographical Information Science* 29, 8, 1470–1489.

Yang, X., Zheng, X.Q., Lv, L.N., 2012. A spatiotemporal model of land use change based on ant colony optimization, markov chain and cellular automata. *Ecological Modelling* 233, 11 – 19.

Yu, J., Chen, Y., Wu, J., 2011. Modeling and implementation of classification rule discovery by ant colony optimisation for spatial land-use suitability assessment. *Computers, Environment and Urban Systems* 35, 4, 308 – 319.

Yue, Q., Xu, X., Hillier, J., Cheng, K., Pan, G., 2017. Mitigating greenhouse gas emissions in agriculture: From farm production to food consumption. *Journal of Cleaner Production* 149, 1011 – 1019.